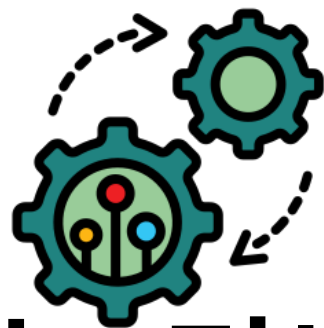




# 케이녹 리버싱 6회차 과제

17기 최동현 (202511688)

# 1-1. 가상 메모리 변환 (VA, RVA, RAW)

## • 가상 메모리 변환

： Windows 운영체제는 실행 파일(PE 포맷)을 디스크에 보관할 때와 프로세스로서 메모리에 적재(Load)할 때 서로 다른 정렬(Alignment) 단위와 주소 체계를 사용한다. 디스크 공간 최적화를 위한 FileAlignment(0x200)와 메모리 페이징 관리를 위한 SectionAlignment(0x1000)의 규격 차이에서 주소의 불일치가 발생한다.

### \* VA, RVA, RAW 특징

- **VA(Virtual Address)** : 프로세스가 실행되어 가상 메모리에 로드되었을 때 시스템이 부여하는 절대적인 메모리 주소
- **RVA(Relative Virtual Address)** : 프로세스의 메모리 기준 주소(ImageBase)로부터 대상 데이터가 얼마나 떨어져 있는지를 나타내는 오프셋이다.  
(VA = ImageBase + RVA 공식을 가짐)
- **RAW (File Offset)** : 프로그램이 디스크 상에 파일(.exe) 형태로 존재할 때 파일의 시작점(0x00)으로부터의 절대적인 물리적 바이트 위치

# 1-2. 가상 메모리 변환 (VA, RVA, RAW)

## • 메모리 주소 연산 구조

```
C memory_addr.c > ...
1  #include <stdio.h>
2  #include <windows.h>
3
4  void calculate_va() {
5      DWORD imageBase = 0x400000; // 4바이트 PE 로드 주소
6      DWORD rva = 0x1000; // .text 섹션 RVA
7
8      // VA 계산
9      DWORD va = imageBase + rva;
10     printf("메모리에 로드된 절대 주소(VA) : 0x%X\n", va);
11 }
12
13 int main(void) {
14     calculate_va();
15     return 0;
16 }
```

```
CHOI@DESKTOP-CHOI Develops
→ ./memory_addr.exe
메모리에 로드된 절대 주소 (VA) : 0x401000
```

- **ASLR(Address Space Layout Randomization) 동적 주소 분석** : 최신 운영체제는 ASLR을 통해 실행마다 ImageBase를 무작위로 변경한다. 따라서 공격자는 익스플로잇 작성 시 절대 주소(VA)를 하드코딩할 수 없기때문에 취약점을 통해 ImageBase를 Leak한 후에 정적으로 분석된 고정값인 RVA를 더해 실행 주소를 RunTime에 계산 해야한다.

# 1-3. 가상 메모리 변환 (VA, RVA, RAW)

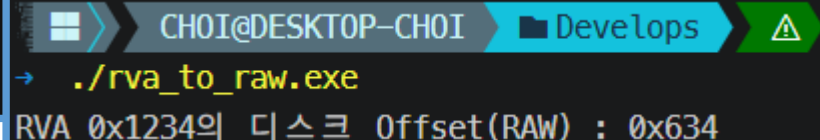
## • RVA -> RAW 복원 (변환 메커니즘)

：정적 분석가(Reverse Engineer)가 디버거에서 발견한 특정 명령어 주소를 Hex Editor를 이용해 디스크의 파일(RAW)에 영구적으로 패치하기 위해 RVA를 RAW로 변환해야한다.

### \* 변환 공식

1. 변환하고자 하는 RVA 값이 속해 있는 메모리 섹션(IMAGE\_SECTION\_HEADER)을 찾는다.
2.  $RAW = RVA - VA(\text{해당 섹션 RVA}) + \text{PointerToRawData}(\text{해당 섹션의 디스크 오프셋})$ 
  - "섹션의 시작점으로부터 떨어진 거리(Offset)는 디스크와 메모리에서 동일하다"는 물리적 원리에 기반

```
4 void rva_to_raw() {
5     DWORD rva = 0x1234; // 리버서가 패치하고자 하는 OEP(Original Entrypoint) RVA
6     DWORD sectionVA = 0x1000; // .text 섹션 메모리 RVA
7     DWORD sectionPointerToRaw = 0x400; // .text 섹션의 디스크 RAW
8
9     // RVA to RAW 변환 공식
10    DWORD raw_offset = rva - sectionVA + sectionPointerToRaw;
11    printf("RVA 0x%X의 디스크 Offset(RAW) : 0x%X\n", rva, raw_offset);
12 }
```



```
CHOI@DESKTOP-CHOI ➤ Develops
➔ ./rva_to_raw.exe
RVA 0x1234의 디스크 Offset(RAW) : 0x634
```

# 1-4. 가상 메모리 변환 (VA, RVA, RAW)

## • ConsoleApplication5.NO\_ASLR.exe (RVA -> RAW 변환)

|                   |                  |   |   |                                 |
|-------------------|------------------|---|---|---------------------------------|
| 00000000140000000 | 0000000000001000 | 사 | 지 | consoleapplication5.no_aslr.exe |
| 00000000140001000 | 0000000000001000 | 사 | 지 | ".text"                         |
| 00000000140002000 | 0000000000002000 | 사 | 지 | ".rdata"                        |
| 00000000140004000 | 0000000000001000 | 사 | 지 | ".data"                         |
| 00000000140005000 | 0000000000001000 | 사 | 지 | ".pdata" rva = 0x1000           |
| 00000000140006000 | 0000000000001000 | 사 | 지 | ".rsrc" rva = va - imageBase    |
| 00000000140007000 | 0000000000001000 | 사 | 지 | ".reloc"                        |

계산기

표현식: mod.offset(140001000)

Hexadecimal: 400

|                |                   |             |                                         |                                    |
|----------------|-------------------|-------------|-----------------------------------------|------------------------------------|
| RIP RAX RDX R9 | 000000001400014F0 | 48:83EC 28  | sub rsp,28                              | OptionalHeader.AddressOfEntryPoint |
|                | 000000001400014F4 | E8 53020000 | call consoleapplication5.no_aslr.140001 |                                    |
|                | 000000001400014F9 | 48:83C4 28  | add rsp,28                              |                                    |
|                | 000000001400014FD | E9 72FEFFFF | jmp consoleapplication5.no_aslr.1400013 |                                    |

VA = 0x1400014F0

RVA = 0x1400014F0 - 0x140000000 = 0x14F0

.text 정보 VA(sectionRVA) = 0x1000, PointerToRawData = 0x400

RAW = 0x14F0 - 0x1000 + 0x400 = 0x8F0

# 2-1. 실행 파일에서의 난독화 구조

## • 난독화(Obfuscation)

： 기계어 코드의 실행 흐름(Control Flow)과 논리 구조를 훼손하지 않으면서 분석 도구와 분석가의 가독성을 극도로 저하시키는 기법이다.

*(핵심은 코드가 암호화되지 않았기때문에 CPU가 별도 복호화 과정 없이 즉각적으로 명령어를 Fetch하고 실행할 수 있다.)*

### \* 난독화 특징

- 제어 흐름 평탄화 (Control Flow Flattening) : 순차적 분기문을 거대한 switch-case 블록으로 묶어 기본 블록(Basic Block) 간의 관계를 단절시킨다.
- 불투명한 술어 (Opaque Predicate) : 실행 전에 무조건 참(True)이거나 거짓(False)임이 확실하지만, 디스어셈블러가 이를 동적 분기로 착각하도록 쓰레기 분기(JZ, JNZ, ==0, !=0 Jump)를 삽입한다.

## 2-2. 실행 파일에서의 암호화 구조

### • 암호화(Encryption)

： 실행 파일 내부의 기계어 코드(.text)나 데이터 영역을 XOR, AES 등의 **수학적 알고리즘**을 통해 전혀 다른 **바이트 배열로 변환**하는 기법입니다.

*(핵심은 암호화된 상태의 바이트들은 CPU가 기계어 명령어(Opcod)로 인식할 수 없는 실행 불가 상태이다. 따라서 프로그램이 정상 작동하기 위해선 복호화를 거쳐야한다.)*

*\* 난독화 암호화 차이*

- 난독화는 암호화가 아니기에 복호화 과정없이 실행(Opcod)가능, 암호화는 복호화 과정을 거치기전까지 실행불가능.

# 3-1. 패킹(Packing)이란? [패커 사용이유]

## • 패킹(Packing)

： 실행 파일 전체를 압축(Compression) 및 암호화(Encryption) 알고리즘으로 묶고, 이를 런타임에 메모리에서 풀어주는 언패킹 스텝(Unpacking Stub)을 원본 파일에 덧붙여 새로운 형태의 PE 실행 파일을 생성하는 기술이다.

*(패커에 의해 바이너리의 PE 헤더 AddressOfEntryPoint는 원래 코드가 아닌 언패킹 스텝의 주소로 덮어쓰워진다.)*

### \* 패킹의 도입 목적

- 용량 최적화 (Compression) : 과거 네트워크 대역폭 부족 시, 바이너리 크기를 줄여 로딩 속도를 높이기 위함.
- 지적 재산 보호 (IP Protection) : 상용 소프트웨어의 소스코드 도용이나 크래킹을 막기 위해 Disassembly를 차단하기 위함.
- 악성코드 은닉 (AV Evasion) : 악성코드의 시그니처 훼손 및 IAT(Import Address Table)를 은닉하여 백신의 정적 탐지률(YARA)을 우회하기 위함.



## 3-2. 패킹 도구(Packer)의 종류, 특징

### 1. 단순 압축 패커 (Compressor / Run-Time Packer)

： 특징으로는 파일 용량 축소 및 로딩 속도 향상이 주 목적이며, 복잡한 암호화나 난독화를 수행하지 않는다.  
(예를 들면 html/js소스가 읽기 쉽게 공백이 채워진 상태가 아니라 공백없이 한 줄의 형태같은 경우)

- 압축 해제 루틴이 간단하여 분석가나 백신이 쉽게 Unpacking 할 수 있다.
- 대표 도구로는 UPX (Ultimate Packer for Executables) 및 ASPack 등이 있다.

### 2. 프로텍터 (Protector / Commercial Packer)

- ： 특징으로는 소프트웨어의 불법 복제 방지 및 러비싱 분석 차단을 목적으로 하는 강력한 보호 도구이다.
- 주요 기능으로는 압축 및 암호화, 난독화, 안티 디버깅, 가상화(원본 기계어를 자체 VM코드 변환) 등 고도화 된 기술을 적용하는 방식으로 한다.
  - 대표 도구로는 VMProtect, Themida, Enigma Protector 등이 있다.

# 3-3. 패커(Packer)의 작동 원리

## • Unpacking Stub & Tail Jump

： 패킹된 프로그램이 실행될 때 OS 로더는 Unpacking Stub을 OEP로 인지하여 실행한다.

### *\* 언패킹 스텝의 핵심 동작 프로세스*

1. **메모리 할당** : 압축이 해제될 원본 코드를 적재할 가상 메모리 공간을 할당한다.
2. **복호화 및 압축 해제** : 패킹된 데이터를 읽어들이 역산(XOR, LZMA 등)을 통해 원본 코드를 복원한다.
3. **IAT 복원** : 패킹으로 인해 OS 로더가 처리하지 못한 원본 프로그램의 API 주소들을 언패킹 스텝이 직접 찾아내어(LoadLibrary, GetProcAddress 등 활용) IAT 테이블을 재구축(Rebuild)하여 원본 코드가 정상적으로 OS 함수를 호출할 수 있도록 한다.
4. **Tail Jump** : 모든 복원이 완료되면, 스텝의 가장 마지막 명령어인 JMP[원본 OEP]를 실행하여 복원된 진짜 코드 영역으로 프로세스 제어권을 양도한다.

# 3-4. 패킹이 완벽 수단이 될 수 없는 이유

- For Hacker

： 아무리 정교한 상용 패커를 사용하더라도, 분석을 완벽히 차단할 수 없는 이유는 컴퓨터의 핵심 구조인 폰 노이만 아키텍처의 태생적 제약 때문이다.

*(폰 노이만 구조에서 CPU가 기계어 명령어를 반입(Fetch)하고 디코딩(Decode)하여 실행하기 위해서는, 해당 기계어가 반드시 주 기억장치(RAM)에 암호화되지 않은 완벽한 평문(Plaintext) 형태로 적재되어 있어야 하기 때문이다.) → CPU는 실시간 데이터를 암복호화가 HW적으로 불가능함, 필연적으로 원본코드를 100% 복호화된 시점이 있다.*

# 3-5. 안티 언패킹 [Stolen Bytes 메커니즘]

## • 스톨른 바이트(Stolen Bytes)

： 메모리 덤프(Memory Dump) 공격을 무력화하기 위해 패커 제작자들이 도입한 고도화된 Anti-Unpacking 기법이다.

*(단순 메모리 복원 및 OEP 점프를 넘어, 원본 프로그램의 OEP에 위치한 기계어 명령어 몇 개를 물리적으로 도난[Stolen]하여 패커의 언패킹 스텝 영역 내부로 은닉하는 기술이다.)*

### *\* Stolen Bytes 흐름*

- 런타임에 언패킹 스텝은 복호화를 마친 뒤 원본 OEP로 바로 점프하지 않고, 자신이 훔쳐 온 명령어들을 스텝 내부에서 직접 실행해버린다.
- 그 후, 도난당한 바이트들의 바로 다음 주소(OEP + Offset)로 점프하여 실행 흐름을 이어간다.



# 감사합니다

17기 최동현